

Module 5 – Structures and Files

Structure: - Structure is a collection of one or more variables of same or different data types grouped to gather under a single name for easy handling. Structure is a user defined data type that can store related information about an entity. Structure is nothing but records about a particular entity.

Declaration of a Structure:- A structure is declared using the keyword struct followed by structure name and variables are declared within a structure. The structure is usually declared before the main() function.

Syntax :-

```
struct structure_name
{
    datatype member 1;
    datatype member 2;
    datatype member 3;
    .....
    .....
    datatype member n;
};
```

Example:-

```
struct employee
{
    int emp_no;
    char name[20];
    int age;
    float emp_sal;
};
```

Initialization of structure :- A structure initialization is done after the declaration of the structure

Syntax :-

```
struct structure_name structure_variable _name;
```

Example :- struct employee emp1;

Accessing structure members :- A structure uses a .(dot) [Member Access Operator] to access any member of the structure.

Example :- emp1.emp_no =12345;

Initialization of structure variables :-

```
struct employee

{
    int emp_no;
    char name[20];
    int age;
    float emp_sal;
} emp1={65421, "Hari",29,25000.00};
```

The order of values enclosed in the braces must match the order of members in the structure definition.

/* C program to read and print the details of employee using structures –static allocation of variables */

```
#include<stdio.h>
#include<string.h>
struct employee
{
    int emp_no;
    char empname[20];
    int age;
    float emp_sal;
};
void main()
{
    struct employee emp1;
    emp1.emp_no = 65421;
    strcpy(emp1.empname,"Hari");
    emp1.age=29;
    emp1.emp_sal=25000.00;
    printf("Employee Number=%d\n",emp1.emp_no);
    printf("Employee Name=%s\n",emp1.empname);
    printf("Employee Age=%d\n",emp1.age);
    printf("Employee Salary=%f\n",emp1.emp_sal);
}
```

/* C program to read and print the details of employee using structures –run time allocation of variables */

```
#include<stdio.h>
#include<string.h>
```

```

struct employee
{
    int emp_no;
    char empname[20];
    int age;
    float emp_sal;
};

void main()
{
    struct employee emp1;
    printf("Enter Employee Number:");
    scanf("%d",&emp1.emp_no);
    printf("Enter Employee Name:");
    scanf("%s",emp1.empname);
    printf("Enter Employee age:");
    scanf("%d",&emp1.age);
    printf("Enter Employee salary:");
    scanf("%f",&emp1.emp_sal);
    printf("Employee Number=%d\n",emp1.emp_no);
    printf("Employee Name=%s\n",emp1.empname);
    printf("Employee Age=%d\n",emp1.age);
    printf("Employee Salary=%f\n",emp1.emp_sal);
}

```

/* C Program to find Sum of Two Complex Numbers*/

```

#include<stdio.h>
struct complex
{
    int real;
    int img; };

void main()
{
    struct complex a,b,c;
    printf("Enter the First Complex Number\n");
    printf("Enter the real part:");
    scanf("%d",&a.real);
    printf("Enter the imaginary part:");
    scanf("%d",&a.img);
    printf("Enter the Second Complex Number\n");
    printf("Enter the real part:");
    scanf("%d",&b.real);
    printf("Enter the imaginary part:");
}

```

```

scanf("%d",&b.img);
c.real=a.real+b.real;
c.img=a.img+b.img;
if(c.img>=0)
printf("Sum of two Complex Number = %d+i%d\n",c.real,c.img);
else
printf("Sum of two Complex Number = %d %di\n",c.real,c.img);
}

```

Array of Structures :- if we want to store the data of 100 employees we would require 100 structure variables from emp1 to emp100 which is definitely impractical the better approach would be to use the array of structures.

/* C program to illustrate array of structures */

```

#include<stdio.h>
#include<string.h>
struct employee
{
    int emp_no;
    char empname[20];
    int age;
    float emp_sal;
};
void main()
{
    struct employee emp1[20];
    int n,i;
    printf("Enter the number of employee entries\n");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("Enter the details of employee %d\n",i+1);
        printf("Enter Employee Number:");
        scanf("%d",&emp1[i].emp_no);
        printf("Enter Employee Name:");
        scanf("%s",emp1[i].empname);
        printf("Enter Employee age:");
        scanf("%d",&emp1[i].age);
        printf("Enter Employee salary:");
        scanf("%f",&emp1[i].emp_sal);
    }
    printf("\nEMP_NO\t EMP_NAME\t EMP_AGE\t\t EMP_SALARY \n");
    for(i=0;i<n;i++)

```

```
    {  
        printf("%d\t%s\t%d\t%f\n",emp1[i].emp_no,emp1[i].empname,emp1[i].age,emp1[i].emp_sal);  
    }  
}
```

Nested Structures :- A nested structure is a structure that contains another structure as its member.

Syntax :-

```
struct structure_name1  
{  
    datatype member 1;  
    datatype member 2;  
};  
struct structure_name2  
{  
    datatype member 1;  
    datatype member 2;  
    struct structure_name1 var1;  
};
```

/* C program to demonstrate nested structures */

```
#include<stdio.h>  
struct stud_dob  
{  
    int day;  
    int month;  
    int year;  
};  
struct student  
{  
    int rollno;  
    char sname[20];  
    struct stud_dob date;  
};  
void main()  
{  
    struct student s;  
    printf("Enter Student Rollno :");  
    scanf("%d",&s.rollno);  
    printf("Enter Student Name :");  
    scanf("%s",s.sname);  
    printf("Enter date of birth as day month year:");  
    scanf("%d%d%d",&s.date.day,&s.date.month,&s.date.year);
```

```
printf("Student Details are\n");
printf("Student Rollno = %d\n",s.rollno);
printf("Student Name=%s",s.sname);
printf("Student DOB= %d-%d-%d\n", s.date.day, s.date.month, s.date.year);
}
```

Structures and Functions :- Structures can be passed to functions and returned from it. Passing structures to functions can be done in the following ways

- Passing individual members
- Passing entire structure or structure variable.

Passing Individual Members :-

while invoking the function from main() in the place of actual arguments we can pass the structure member as an argument.

```
#include<stdio.h>
void add(int x,int y);
struct addition
{
    int a;
    int b;
};
void main()
{
    struct addition sum;
    printf(" Enter two numbers\n");
    scanf("%d%d",&sum.a,&sum.b);
    add(sum.a,sum.b);
}
void add(int x,int y)
{
    int res;
    res=x+y;
    printf("Resultant=%d\n",res);
}
```

Passing Entire structure or structure variable :-while invoking the function instead of passing the individual members the entire structure, i.e. the structure variable is passed as a parameter to the function.

```
#include<stdio.h>
struct addition
{
    int a;
    int b;
```

```

};
void add(struct addition sum);
void main()
{
    struct addition sum;
    printf(" Enter two numbers\n");
    scanf("%d%d",&sum.a,&sum.b);
    add(sum);
}

```

Union:- A union is a special data type available in C that allows to store different data types in the same memory location. You can define a union with many members, but only one member can contain a value at any given time. Unions provide an efficient way of using the same memory location for multiple-purpose.

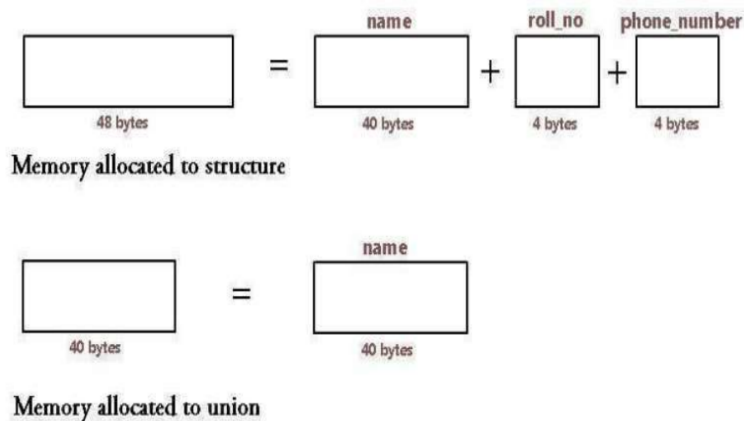
Syntax for declaring a union is same as that of declaring a structure except the keyword struct.

```

union union_name {
    datatype field_name;
    datatype field_name;
    // more variables
};

```

	Structure	Union
Keyword	The Keyword struct is used to define the Structure	The keyword union is used to define a union.
Size	When a variable is associated with a structure, the compiler allocates the memory for each member. The size of structure is greater than or equal to the sum of sizes of its members.	when a variable is associated with a union, the compiler allocates the memory by considering the size of the largest memory. So, size of union is equal to the size of largest member.
Memory	Each member within a structure is assigned unique storage area of location.	Memory allocated is shared by individual members of union.
Value Altering	Altering the value of a member will not affect other members of the structure.	Altering the value of any of the member will alter other member values.
Accessing members	Individual member can be accessed at a time.	Only one member can be accessed at a time.
Initialization of Members	Several members of a structure can initialize at once.	Only the first member of a union can be initialized.



Arrays of Union Variables

We can create array of unions similar to creating array of any primitive data type. general form of declaration for array of union.

union <union_name><array_name>[size];

Consider the following example,

```
union values
{
    int int_val;
    float float_val;
};
union values arr[2];
```

Here, arr is an array of union which can hold two union elements.

Array of union initialization:

union values arr[2] = {{1}, {2}};

```
#include <stdio.h>

union values {
    int integer_val; // union member    };
int main() {
    union values arr1[2] = {{10}, {20}};
    printf("arr1[0].integer_val: %d\n", arr1[0].integer_val);
    printf("arr1[1].integer_val: %d\n", arr1[1].integer_val);
    printf("Sizeof(union values): %d\n", sizeof(union values));
    return 0;
}
```


Structures Inside Unions :- A structure can be nested inside a union and it is called union of structures.

Enumerated Data Type:-

- Enumeration or Enum in C is a special kind of data type defined by the user.
- Enumerated data consists of named integer constants as a list.
- Its value by default starts from zero and incremented by one sequentially.
- Syntax: **enum identifier{enumerated list};**

```
#include<stdio.h>
```

```
enum days{Sunday=1, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday};
```

```
void main()
```

```
{  
    for(int i=Sunday ; i<=Saturday; i++)  
    {  
        printf("%d, ",i);  
    }  
}
```

Files

A file is a collection of data stored on secondary storage device like hard disk.

Types of Files:

1. ASCII Text files
2. Binary files

1. ASCII Text files:-

Text files are the normal .txt files. You can easily create text files using any simple text editors such as Notepad. When you open those files, you'll see all the contents within the file as plain text. You can easily edit or delete the contents.

2. Binary files:-

Binary files are mostly the .bin files in your computer. Instead of storing data in plain text, they store it in the binary form (0's and 1's).

File Operations

1. Creating a new file
2. Opening an existing file
3. Closing a file
4. Reading from and writing information to a file

1. Creating a new file:

When working with files, you need to declare a pointer of type FILE. This declaration is needed for communication between the file and the program.

Syntax:- FILE * fptr;

2. Opening an existing file:

- A file must first be opened before reading/writing data to it.
- Opening a file is performed using the fopen() function defined in the stdio.h header file.
 - Syntax- FILE fptr;
 fptr= fopen("filename", "mode");
- Various modes- r : read mode, w: write mode , a: append to text file

3. Closing a file:

The file (both text and binary) should be closed after reading/writing. Closing a file is performed using the fclose() function.

fclose(fptr);

File Operations - open and close mode:

```
#include <stdio.h>

int main() {
    // Declare a file pointer
    FILE *file;
    // Open the file for reading
    file = fopen("myfile.txt", "r");
    // Check if the file was opened successfully
    if (file == NULL) {
        printf("Failed to open the file.\n");
        return 1; // Return an error code
    } // Read and print the content of the file
    char character;
    while ((character = fgetc(file)) != EOF) {
        putchar(character);
    } // Close the file
    fclose(file);
    return 0; // Return success
}
```

File Operations – Write mode:

```
#include <stdio.h>
int main() { // Open for writing
    FILE *file = fopen("myfile1.txt", "w");

    if (file == NULL) {
        perror("Error opening file");
        return 1;
    }
    fprintf(file, "Hello, File!\n");
    fputs("Another line.\n", file);
    fclose(file);
    return 0;
}
```

Reading data from files:

- fscanf()- Used to read formatted data from file.
Syntax: fscanf(fptr,format_specifier,address_list);
- fgets()- Used to read a string from file.
Syntax: fgets(string_name, n, fptr);
- fgetc()- Used to read a single character from file.
Syntax: char ch= fgetc(fptr);

Writing data to files:

- fprintf()- Used to write formatted output to file.
Syntax- fprintf(fptr, format_specifier,variable_list);
- fputs()- Used to write a line/string to file.
Syntax- fputs(string_name, fptr);
- fputc()- Used to write a character to file.
Syntax- fputc(c,fptr);

Detecting the End of File :

- "End of file" (EOF) is a term used in computer programming and operating systems to indicate the point in a file where there is no more data to be read.
- When a program reads data from a file, it typically keeps reading until it encounters the end of file marker, at which point it knows that there is no more data to be processed.
- The function eof() is used to check the end of file after EOF mark.

```
#include <stdio.h>
int main() {
    FILE *file = fopen("myfile.txt", "r"); // Open the file for reading
    if (file == NULL) {
        perror("Error opening file");
        return 1;
    }
    int ch;
    while ((ch = fgetc(file)) != EOF) {
        putchar(ch); // Print each character from the file
    }
    fclose(file);
    return 0;
}
```